EOS Assignments – Command Cheat Sheet

Nov 12, 2026 · @CDAC_Ganesh

Every command needed for Assignments 1–4, each with a two-line meaning; PID means the process ID shown by ps.

Common setup (used in every assignment)

Command	Meaning (two lines)
`gcc while1.c -o while1.out`	Compiles the C source into an executable. Add `-lrt` (and `-pthread`) when the program uses shared memory or threads.
`./while1.out &`	Starts the program in the background; the shell prints its job number and PID. The terminal stays free for ps, top and renice.
`taskset -c 1 ./while1.out &`	Starts the program pinned to CPU 1 only, so all instances compete on one core. Used in Assignment 1 so the CPU shares are easy to read.
`taskset -cp 1 <PID>`	Pins an already running process to CPU 1. `taskset -p <PID>` only shows its current CPU mask.
`nproc` / `lscpu`	Show how many CPUs the machine has. Helps you choose a valid CPU number for taskset.
`ps -eo pid,ppid,cmd,cls,ni,pri,rtprio,stat,psr,pcpu | grep while1`	Lists instances with scheduling class (TS, FF, RR), nice, priority, state, CPU number and CPU %. The main observation command for Assignments 1 and 3.
`top`	Live view of CPU share per process (PR, NI, S, %CPU, TIME+). Press `1` for per-CPU lines, `q` to quit; `top -p <PID1>,<PID2>` watches only chosen PIDs.

Command	Meaning (two lines)
`uptime`	Prints the load average for the last 1, 5 and 15 minutes. It rises with the number of runnable processes.
`jobs`	Lists background jobs started from this shell. `fg %1` brings one to the front and `Ctrl+C` stops it.
`kill <PID>` / `pkill while1.out`	Terminates one process, or every process with that name. Run it at the end of each experiment to clean up.
`sudo`	Runs one command with root rights. Use it only for negative nice, real-time chrt and writing under /proc/sys.

Assignment 1 – Process scheduling and real-time scheduling

Q1. Process loading and observation (steps 1–5)

Command	Meaning (two lines)
`gcc while1.c -o while1.out`	Step 1: compiles the infinite-loop program. The loop never sleeps, so each instance always wants the CPU.
`taskset -c 1 ./while1.out &` (run 2 times)	Step 2: starts two pinned background instances. Both compete for CPU 1, so each gets about 50%.
`ps -eo pid,cmd,cls,stat,psr | grep while1` and `top`	Step 3: shows state R, class TS, PSR 1 and PR 20 / NI 0 for every instance. Equal weights give an equal CPU share.
`taskset -c 1 ./while1.out &` (2 more, then 2 more)	Steps 4–5: raises the count to 4 and then 6 instances. Share per process drops to about 25% and then about 16.6% (100 ÷ N), and the load average rises.

Q2. Changing nice values (steps 6–10)

Command	Meaning (two lines)
`renice -n 5 -p <PID>`	Sets the nice value of a running process to +5 (also +10, +15). A positive value lowers priority (PR = 20 + NI) and shrinks its CPU share; any user may raise nice.
`renice -n 10 -p <PID1> <PID2>`	Changes several processes in one command. Useful to compare different nice levels side by side in top.
`nice -n 10 taskset -c 1 ./while1.out &`	Starts a new process already at nice +10. Same effect as renice, but applied at launch.
`sudo renice -n -5 -p <PID>`	Step 10: sets a negative nice (higher priority, PR 15). Only root can lower nice; the process gets a much bigger share.
`sudo renice -n -20 -p <PID>`	Sets the highest normal priority (PR 0). That one process takes about 95% of the CPU and starves the others; use with caution.
`ps -o pid,ni,pri,pcpu,cmd -p <PID>`	Shows nice (NI), priority (PRI) and CPU % of chosen processes. Use it, or `top`, after every renice to record the change.
`sudo renice -n 0 -p <PID>`	Restores the default nice value. Do this (or kill the processes) when the experiment ends.

Q3. Changing scheduling policy with chrt (steps 11–15)

Command	Meaning (two lines)
`chrt -m`	Prints the minimum and maximum priority for every policy (FIFO and RR use 1–99, OTHER uses 0). Check it first so you pick valid priorities.
`chrt -p <PID>`	Shows the current policy and priority of a process. Use it to verify that a change really happened.

Command	Meaning (two lines)
`sudo chrt -f -p 10 <PID>`	Step 11: switches a running process to SCHED_FIFO with real-time priority 10 (`-f` = FIFO). It runs until it blocks or yields, so it can hog the CPU.
`sudo chrt -r -p 10 <PID>`	Switches a running process to SCHED_RR priority 10 (`-r` = round robin). Equal-priority RR processes take turns with a time slice (about 48% each).
`sudo chrt -o -p 0 <PID>`	Puts the process back to SCHED_OTHER, shown as TS (`-o`). Use it to restore processes after the experiment.
`sudo chrt -f 10 taskset -c 1 ./while1.out &`	Launches a new process directly as FIFO priority 10 on CPU 1. Faster than changing a running process.
`sudo chrt -f -p 10 <PID1>` and `sudo chrt -f -p 10 <PID2>`	Step 12: two equal-priority FIFO processes next to TS processes on the same CPU. The first FIFO one takes about 95% and the second gets almost nothing.
`sudo chrt -r -p 10 <PID1>` and `sudo chrt -r -p 10 <PID2>`	Step 13: two equal-priority RR processes pinned to one CPU. They alternate, so each gets about 48%.
`sudo chrt -f -p 10 <PID1>` and `sudo chrt -f -p 2 <PID2>`	Step 14: FIFO with different priorities on one CPU. Priority 10 gets about 95%; priority 2 is starved.
`taskset -cp 1 <PID>`	Pins the chosen processes to the same CPU before the chrt tests. Real-time effects are only visible when the processes share one CPU.
`ps -eo pid,cmd,cls,rtprio,ni,pri,pcpu,psr | grep while1`	Shows class (TS, FF, RR), RT priority and CPU %. In top an RT process shows PR = -(1 + rtprio), e.g. -11 for priority 10.

Command	Meaning (two lines)
`gcc while1_yield.c -o while1_yield.out`	Step 15: compiles the copy of while1.c that calls `sched_yield()` inside the loop. Run it as FIFO; the yielding FIFO processes share the CPU about 48% each.
`cat /proc/sys/kernel/sched_rt_runtime_us`	Shows the real-time throttle (950000 µs per 1000000 µs). This is why an RT hog is capped at about 95% and the system stays usable.
`cat /proc/sys/kernel/sched_rr_timeslice_ms`	Shows the RR time slice in milliseconds (default about 100). It is the time an RR process runs before the next equal-priority one gets a turn.

Safety: do real-time tests only on a test machine, keep a spare terminal open, and finish with `sudo chrt -o -p 0 <PID>` or `pkill while1`.

Assignment 2 – Virtual memory and address space

Q1. ASLR and address-space access (steps 16–23)

Command	Meaning (two lines)
`cat /proc/sys/kernel/randomize_va_space`	Shows the ASLR mode: 0 = off, 1 = stack/libraries random, 2 = full (default). Note the original value so you can restore it.
`echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`	Turns ASLR off system-wide. Addresses now repeat on every run, which is handy for debugging.
`echo 2 | sudo tee /proc/sys/kernel/randomize_va_space`	Turns full ASLR back on (the default). Always restore it after the experiment.
`sudo sysctl -w kernel.randomize_va_space=0`	Same as the echo command, written with sysctl. Use `=2` to restore.
`setarch $(uname -m) -R ./app`	Runs just this one program with ASLR disabled (`-R`). No system setting is changed, so it is the safest option.

Command	Meaning (two lines)
`gcc app.c -o app`	Compiles the test program that prints addresses of a function (code), a const string (read-only data), a global (data), a heap block and a stack variable. Add `-no-pie` to get a fixed code address.
`./app &` (run several times)	Starts multiple instances, with ASLR on and then off. Compare the printed addresses: they change with ASLR on and stay equal with ASLR off.
`cat /proc/<PID>/maps`	Lists every memory region of the process with its address range and permissions (r-xp code, r--p read-only data, rw-p data/heap/stack). Match the printed addresses to these ranges (step 18).
`pmap -x <PID>`	Shows the same regions in a table with sizes, RSS and permissions. Easier to read than maps for a quick check.
`readelf -S app` / `readelf -l app`	Show the ELF sections and segments (.text, .rodata, .data, .bss) with their flags. Confirms which permissions each part should have.
`size app`	Prints the size of the text, data and bss areas. A quick way to see the program's static layout.
`./app 0x<ADDR>` (program that reads or writes an address given on the command line)	Steps 19–22: reads or writes a code, read-only, data and an unmapped address. Reading code or data works; writing to code or read-only data, or touching an unmapped address, gives a segmentation fault.
`echo $?`	Prints the exit status of the last command. A crash by SIGSEGV shows 139 (128 + signal 11).
`dmesg | tail` (or `sudo dmesg | tail`)	Shows the kernel's segfault message with the faulting address and instruction. It is useful evidence for step 23.

Command	Meaning (two lines)
`gdb ./app`	Optional debugger to run the program and see the faulting address, using `run`, `bt` and `info proc mappings`. Not required, but explains why the fault happened.

Q2. Memory overcommit experiment (steps 24–28)

Command	Meaning (two lines)
`gcc example2.c -o ex2`	Step 24: compiles the provided program that allocates the number of MB given as its argument. It only reserves virtual memory and never touches it.
`cat /proc/sys/vm/overcommit_memory`	Shows the overcommit mode: 0 = heuristic, 1 = always allow, 2 = strict. Note the original value (usually 0) before changing it.
`cat /proc/sys/vm/overcommit_ratio`	Shows the percentage of RAM counted as the limit in mode 2 (default 50). Record it as part of the system configuration (step 28).
`free -h`	Shows total RAM, used memory and swap in readable units. Record it to compare physical memory with the VSZ you reach.
`echo 1 | sudo tee /proc/sys/vm/overcommit_memory`	Step 25: sets mode 1 so the kernel accepts every allocation request. It lets you allocate far more virtual memory than the RAM installed.
`./ex2 10 &` `./ex2 100 &` `./ex2 1000 &` `./ex2 2000 &`	Step 26: starts several instances requesting 10, 100, 1000 and 2000 MB. Try larger values (10000, 100000) to push the total higher.
`ps -eo pid,vsz,rss,cmd | grep ex2`	Step 27: shows VSZ (virtual size) and RSS (real RAM used) per instance. VSZ matches the request while RSS stays tiny, because the pages are never touched.

Command	Meaning (two lines)
`ps -C ex2 -o vsz= | awk '{s+=$1} END {print s/1024 " MB"}'`	Adds up the VSZ of all ex2 processes and prints the total in MB. This is the number to report as your largest total VSZ.
`top`	Shows VIRT and RES columns for the processes. Use it to confirm the big VIRT but small RES.
`grep -i commit /proc/meminfo`	Shows CommitLimit and Committed_AS (memory promised to processes). Committed_AS far above CommitLimit proves the system is overcommitting.
`echo 0 | sudo tee /proc/sys/vm/overcommit_memory`	Restores the original mode (use your recorded value). Do this after testing, as the setting is system-wide.
`pkill ex2`	Stops all ex2 processes. Frees the reserved address space.
`sudo dmesg | grep -i -E "oom|killed"`	Checks whether the OOM killer terminated anything. Needed only if you touch the memory and RAM runs out.

Assignment 3 – Process creation, termination and waitpid()

Q1. Creating and cleaning up child processes (steps 29–34)

Command	Meaning (two lines)
`gcc parent.c -o parent`	Compiles the parent program that forks five children and calls waitpid() only after creating all of them. Children exit with 0, with a non-zero code, or loop forever.
`./parent &`	Runs the parent in the background so you can watch it from another terminal. It prints each child's PID and its decoded status.
`ps -o pid,ppid,stat,cmd --ppid <PARENT_PID>`	Lists only the children of the parent with their state. R = running, S = sleeping, Z = zombie (finished but not yet collected).

Command	Meaning (two lines)
`ps -eo pid,ppid,stat,cmd | grep -E "parent|defunct"`	Finds zombie children, shown as `<defunct>` with state Z. They exist when children end before the parent calls waitpid().
`top`	Shows the process states and the Zombie count in the header. Use it with ps as step 34 asks.
`kill -SIGKILL <CHILD_PID>` (or `kill -9 <CHILD_PID>`)	Step 33: kills a looping child abnormally. The parent's waitpid() then reports termination by signal 9.
`kill -SIGTERM <CHILD_PID>`	Asks a child to terminate politely (signal 15). The status shows signal 15 instead of an exit code.
`echo $?`	Prints the exit status of the last shell command. 0 = success, non-zero = failure, 128 + n = killed by signal n.
`WIFEXITED(st)` / `WEXITSTATUS(st)`	Inside the parent: true if the child ended normally, and then gives its exit code. Code 0 = normal success, any other code = normal failure (step 32).
`WIFSIGNALED(st)` / `WTERMSIG(st)`	Inside the parent: true if the child was killed by a signal, and then gives the signal number. This is the abnormal termination case (for example 9 for SIGKILL, 11 for SIGSEGV).
`fork()` / `waitpid(pid, &st, 0)`	`fork()` creates a child and returns 0 in the child and the child's PID in the parent. `waitpid()` blocks until that child ends and fills in its status.

Q2. Launching applications from child processes (steps 35–39)

Command	Meaning (two lines)
`gcc launcher.c -o launcher`	Compiles the parent that forks five children, each calling an exec function. The parent calls waitpid() for all of them afterwards.
`execlp("kwrite", "kwrite", NULL)` / `execlp("evince", "evince", "file.pdf", NULL)`	Replaces the child with the named application, searched in PATH (kwrite as editor, evince or okular as PDF viewer). Close the window normally for exit 0, or kill it for abnormal termination.
`execlp("gcc", "gcc", "hello.c", "-o", "hello", NULL)`	Runs the compiler in the child. A valid file gives exit status 0; a file with errors gives exit status 1 (normal but unsuccessful).
`execl("./while1.out", "while1.out", NULL)`	Runs the infinite-loop program in the child. It never ends by itself, so end it with `kill -SIGKILL <PID>` and the parent reports termination by signal.
`execlp("nonexistent", ...)` then `exit(127)`	If exec fails, the child continues and should exit with a non-zero code. This shows an unsuccessful normal termination.
`which kwrite evince okular gcc`	Checks that the applications exist on your system. Use the viewer that is installed.
`ps -o pid,ppid,stat,cmd --ppid <PARENT_PID>`	Shows each child after exec with its new command name. Confirms which application each child is running.
`kill -SIGKILL <CHILD_PID>`	Terminates one launched application abnormally. The status is decoded with WIFSIGNALED and WTERMSIG.

Q3. Parallel compilation and linking (steps 40–44)

Command	Meaning (two lines)
`gcc -c file1.c -o file1.o` (files 1 to 5)	Compiles one source file into an object file without linking. Each of the five children runs one of these through `execlp("gcc", "gcc", "-c", ...)`.
`gcc file1.o file2.o file3.o file4.o file5.o -o final`	Links all object files into the final executable. The parent runs it only if every child returned exit status 0.
`gcc parallel.c -o parallel`	Compiles the parent program that forks the five compile children, collects them with waitpid(), then links and runs.
`ls -l *.o`	Lists the object files that were generated. Confirms which children really succeeded.
`./final`	Runs the linked binary. In the parent this is a fork and exec of `./final` as a new process.
`echo $?`	Shows the exit status of the last compile, link or run. Non-zero means that step failed.
`make -j5` (optional)	Does the same parallel compile with make, using five jobs. Only for comparison; the assignment needs fork and waitpid.
`nm file1.o` (optional)	Lists symbols in an object file. Helps explain a link error such as an undefined reference.

Assignment 4 – Shared memory

Q1. Shared-memory communication (steps 45–50)

Command	Meaning (two lines)
`getconf PAGESIZE`	Prints the page size in bytes (normally 4096). Three pages = 3 × this value, the size to give the shared object.

Command	Meaning (two lines)
`gcc shm_parent_child.c -o shm_pc -lrt`	Compiles the parent-child program (steps 45–47). `-lrt` links the POSIX shared-memory functions on older systems.
`gcc writer.c -o writer -lrt` and `gcc reader.c -o reader -lrt`	Compile the two unrelated programs (steps 48–49). They attach to the same object through its name.
`./writer` in terminal 1, then `./reader` in terminal 2	Runs two independent processes that share data. The reader prints what the writer put in each of the three pages.
`ls -l /dev/shm`	Lists POSIX shared-memory objects (files named like `/myshm`). Shows that the object exists and its size (3 × page size).
`cat /proc/<PID>/maps | grep -i shm`	Shows the shared region mapped into one process (look for `/dev/shm/...` with `rw-s`). Run it for both processes to prove they attach to the same object (step 50).
`pmap -x <PID>`	Lists the process mappings with sizes. Confirms the 12 KB shared region is attached.
`ipcs -m`	Lists System V shared-memory segments with key, id, owner, size and number of attachments. Use it if you use shmget instead of shm_open.
`ipcrm -m <shmid>`	Removes a System V segment by id. Clean up after the test.
`rm /dev/shm/<name>`	Deletes a POSIX shared-memory object left behind. The program normally does this with `shm_unlink()`.
`shm_open("/myshm", O_CREAT | O_RDWR, 0666)`	Creates or opens the named shared-memory object and returns a file descriptor. The name is how unrelated processes find the same object.
`ftruncate(fd, 3 * pagesize)`	Sets the object size to three pages (step 45). The size is zero until you do this.

Command	Meaning (two lines)
`mmap(NULL, 3 * pagesize, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)`	Maps the object into the process and returns a pointer. `MAP_SHARED` makes writes visible to every other process that maps it.
`fork()`	Creates the child after the mapping exists, so the child inherits the same shared mapping. It can read what the parent wrote in all three pages (step 47).
`munmap(ptr, size)` / `close(fd)` / `shm_unlink("/myshm")`	Detach the region, close the descriptor, and delete the object name. Call `shm_unlink` once, in one process, when finished.
`shmget` / `shmat` / `shmdt` / `shmctl(IPC_RMID)`	The System V alternative: create a segment, attach it, detach it, and remove it. Use these instead of shm_open and mmap only if your lectures used System V.